Kate Lee

February 28, 2026

IT FDN 110 GP

Assignment 05

JSON, Dictionaries, & If At First You Don't Succeed, Try-Except Again

Introduction

Module 5 upgrades the course registration program by swapping "flat" data storage like comma-separated values for a more structured approach: a list of dictionaries stored in a JSON file. The program also includes structured error handling using try-except so the program can recover from silly mistakes a user might make instead of face-planting and quitting. For this assignment, I modified the starter program to load registrations from a JSON file, let the user register additional students, display the current registrations, and save updates back to the file, all while checking for various exceptions/errors.

Steps I Took

1. Reviewed the Starter File and Acceptance Criteria

I used the provided starter script and compared it line-by-line to the acceptance criteria. The starter version used CSV-style lists, so I updated it to use dictionaries and JSON as required (see image below).

```python

# Define the Data Variables
student_first_name: str = ""
student_last_name: str = ""
course_name: str = ""
file = _io.TextIOWrapper  # Allows us to use file.close() in the finally block
menu_choice: str = ""
student_data: dict = {}   # one row of student data (dictionary)
students: list = []       # a table of student data (list of dictionaries)

```

2. Switched Data Structure to a List of Dictionary Rows

Instead of storing each registration as a list like ['Bob','Smith','Python 100'], I stored each registration as a dictionary with keys 'FirstName', 'LastName', and 'Course'. This structure aligns naturally with JSON (JavaScript Object Notation), since JSON objects map directly to Python dictionaries. It also makes the data more readable, because the keys act like column names in a spreadsheet rather than relying on numeric index positions.

3. Loaded Starting Data from Enrollments.json

When the program starts, it opens Enrollments.json and loads the contents into the students table using json.load(). This makes the program immediately useful instead of starting with a bunch of nothing. The image to the right shows the JSON I have in my starter Enrollments.json file.

Assignment05.py | Enrollments.json

```
[
  {"FirstName": "Kate",
    "LastName": "Lee",
    "Course": "Python 100"
  },
  {"FirstName": "Jeong",
    "LastName": "Lee",
    "Course": "Python 200"
  },
  {"FirstName": "Chae",
    "LastName": "Lee",
    "Course": "Python 300"
  }
]
```

What I'd really like to figure out is how to get these JSON objects to automatically format to look nice and spaced out like that. I had to do this by hand to get that screenshot, but I'm sure I'll figure it out eventually!

4. Added a Menu Loop that Supports Multiple Registrations

The program runs inside a while loop and repeatedly shows a menu. Users can register multiple students, view all current registrations, and save them back to the JSON file.

5. Implemented Structured Error Handling (Try-Except)

I added try-except blocks in four key places required by the assignment:

1) Upon reading the file at startup, to ensure it exists. If it does not, the program continues running and the file will be created later when data is saved.
2) When entering the first name, to make sure only alphabetic characters were used.
3) When entering the last name, to validate the same requirement.
4) When writing the updated data back to the file, to display any errors and to ensure the file is properly closed so it doesn't linger in memory like a nuisance.

This follows the module's guidance that try-except is the recommended pattern for user-friendly and organized error handling.

6. Verified Output Formatting (Comma-Separated Display)

For menu choice 2 (and after saving in menu choice 3), the program prints each registration as both a readable message and a line of comma-separated values. Even though the data is stored in JSON format, the display output intentionally matches the assignment's two required styles, using f-strings to format each:

```
    # Present the current data
    elif menu_choice == "2":

        # Process the data to create and display a custom message
        print("-" * 50)
        for student in students:
            print(f"Student {student['FirstName']} {student['LastName']} is enrolled in {student['Course']}")
        print("-" * 50)

        # Also show comma-separated values for each row
        for student in students:
            print(f"{student['FirstName']},{student['LastName']},{student['Course']}")
        print("-" * 50)
        continue
```

Source Control

GitHub Repository Link: https://github.com/katherinelee703/IntroToPython

Summary

This assignment helped me understand the difference between lists and dictionaries, how JSON maps naturally to Python collections, and why structured error handling matters. The finished program loads existing registrations from Enrollments.json (if there are any), lets the user add more entries, displays the current registrations, and saves updates back to the JSON file. Now the program feels robust enough to survive a moderate degree of reality, including typos and missing files. Honestly, I keep thinking of Murphy's Law – "anything that can go wrong, will" – while doing my assignments, so I'm glad we gained more experience using try-except-finally blocks this week.

Works Cited

OpenAI. (2026). ChatGPT (GPT-5.2 Thinking). Used for code troubleshooting and written outline assistance.

University of Washington. (2026). Introduction to Programming with Python: Module 05 Notes.

University of Washington. (2026). Introduction to Programming with Python: Module 05 Assignment Instructions.